

Workshop 11: Advanced SQL Queries

SQLite CRUD and Full-Stack Database Integration

เป้าหมาย

Workshop นี้ให้นักศึกษาฝึกนำ SQL มาใช้งานร่วมกับ Frontend และ Express.js โดยใช้ SQLite เป็นฐานข้อมูล

เนื้อหาหลักประกอบด้วย

- Express Static Files
- Fetch API
- SQLite Database
- INSERT และ SELECT
- ORDER BY
- UPDATE และ DELETE
- Route Parameter
- Complete CRUD Flow

Workshop ประกอบด้วย 3 งาน

1. Circle Calculator with Express
2. Save and Read Data with SQLite
3. Update and Delete Data

งานที่ 1 — Circle Calculator with Express

สร้าง Web Application สำหรับคำนวณพื้นที่วงกลม โดยแยก Frontend และ Backend โครงสร้าง Project

```
sql2-demo/  
|-- app.js  
|-- package.json  
|  
`-- public/  
    |-- index.html  
    |-- style.css  
    `-- script.js
```

Step 1 — สร้าง Project

เปิด Terminal แล้วพิมพ์

```
mkdir sql2-demo
cd sql2-demo
npm init -y
npm install express
```

Step 2 — สร้าง Express Server

สร้างไฟล์ app.js

```
const express = require("express");

const app = express();
const port = 3000;

app.use(express.json());

app.use(
  express.static("public")
);

app.post("/calculate", (req, res) => {

  const radius = Number(req.body.radius);

  const area = Math.PI * radius * radius;

  res.json({
    radius: radius,
    area: area
  });

});

app.listen(port, () => {

  console.log(
    "Server started at http://localhost:3000"
  );

});
```

```
});
```

Step 3 — สร้างหน้าเว็บ

ไฟล์ public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Circle Calculator</title>

  <link rel="stylesheet"
        href="style.css">
</head>

<body>

  <h1>Circle Calculator</h1>

  <input
    type="number"
    id="radius"
    placeholder="Enter radius">

  <button onclick="calculate()">
    Calculate
  </button>

  <p id="result"></p>

  <script src="script.js"></script>

</body>
</html>
```

Step 4 — ส่งข้อมูลด้วย fetch()

ไฟล์ public/script.js

```
async function calculate() {

  const radius =
```

```
document.getElementById("radius").value;

const response =
  await fetch("/calculate", {

    method: "POST",

    headers: {
      "Content-Type":
        "application/json"
    },

    body: JSON.stringify({
      radius: radius
    })

  });

const data =
  await response.json();

document.getElementById("result").innerText =
  "Area = " + data.area.toFixed(2);
}
```

ทดสอบ

Run Server

```
node app.js
```

เปิด

<http://localhost:3000>

ทดลองกรอก

radius = 5

ผลลัพธ์ควรได้ประมาณ

Area = 78.54

งานที่ 2 — Save and Read Data with SQLite

ปรับโปรแกรมให้บันทึกผลการคำนวณ ลงฐานข้อมูล SQLite

Step 1 — ติดตั้ง SQLite

เปิด Terminal

```
npm install sqlite3
```

Step 2 — เชื่อมต่อ Database

เพิ่มใน app.js

```
const sqlite3 =  
  require("sqlite3").verbose();  
  
const db =  
  new sqlite3.Database("circle.db");
```

Step 3 — สร้าง Table

เพิ่ม

```
db.run(`  
  CREATE TABLE IF NOT EXISTS circles (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    radius REAL,  
    area REAL  
  )  
`);
```

Table circles ประกอบด้วย

- id — Primary Key
- radius — รัศมี
- area — พื้นที่

Step 4 — INSERT ผลการคำนวณ

แก้ Route /calculate

```

app.post("/calculate", (req, res) => {

  const radius =
    Number(req.body.radius);

  const area =
    Math.PI * radius * radius;

  const sql =
    "INSERT INTO circles (radius, area)
    VALUES (?, ?)";

  db.run(
    sql,
    [radius, area],

    function (err) {

      if (err) {

        return res.status(500).json({
          error: "Database error"
        });

      }

      res.json({
        id: this.lastID,
        radius: radius,
        area: area
      });
    }
  );
});

```

เครื่องหมาย

?, ?

ใช้เป็น Parameter แทนการนำค่าจากผู้เข้ามาต่อกับ SQL โดยตรง

Step 5 — อ่าน History

สร้าง Route

```
app.get("/history", (req, res) => {

  const sql =
    "SELECT * FROM circles
    ORDER BY id DESC";

  db.all(sql, [], (err, rows) => {

    if (err) {

      return res.status(500).json({
        error: "Database error"
      });

    }

    res.json(rows);

  });

});
```

ORDER BY

คำสั่ง

```
ORDER BY id DESC
```

หมายถึง เรียงข้อมูลจาก รายการใหม่ไปหารายการเก่า

Step 6 — แสดง History บนหน้าเว็บ

เพิ่มใน index.html

```
<h2>Calculation History</h2>

<button onclick="loadHistory()">
  Load History
</button>
```

```
<ul id="history"></ul>
```

เพิ่มใน script.js

```
async function loadHistory() {

  const response =
    await fetch("/history");

  const data =
    await response.json();

  let html = "";

  data.forEach(item => {

    html +=
      "<li>" +
      "Radius = " + item.radius +
      ", Area = " +
      item.area.toFixed(2) +
      "</li>";

  });

  document.getElementById("history").innerHTML =
    html;
}
```

งานที่ 3 — Update and Delete Data

เพิ่มความสามารถในการแก้ไข และลบข้อมูลจาก Database

Step 1 — DELETE

สร้าง Route

```
app.delete("/delete/:id", (req, res) => {

  const id =
    req.params.id;

  const sql =
    "DELETE FROM circles
    WHERE id = ?";

  db.run(sql, [id], function (err) {

    if (err) {

      return res.status(500).json({
        error: "Delete failed"
      });

    }

    res.json({
      success: true
    });

  });

});
```

Frontend DELETE

เพิ่ม Function

```
async function deleteItem(id) {

  await fetch(
```

```

        "/delete/" + id,
        {
            method: "DELETE"
        }
    );

    loadHistory();
}

```

หลังจากลบข้อมูลแล้ว ให้เรียก loadHistory() เพื่อ Refresh รายการ

Step 2 — UPDATE

สร้าง Route

```

app.put("/update/:id", (req, res) => {

    const id =
        req.params.id;

    const radius =
        Number(req.body.radius);

    const area =
        Math.PI * radius * radius;

    const sql =
        `UPDATE circles
        SET radius = ?, area = ?
        WHERE id = ?`;

    db.run(
        sql,
        [radius, area, id],

        function (err) {

            if (err) {

                return res.status(500).json({
                    error: "Update failed"
                });
            }
        }
    );
}

```

```

        }

        res.json({
            success: true
        });
    }
};

});

```

Frontend UPDATE

ตัวอย่าง Function

```

async function editItem(id) {

    const radius =
        prompt("Enter new radius:");

    await fetch(
        "/update/" + id,
        {
            method: "PUT",

            headers: {
                "Content-Type":
                    "application/json"
            },

            body: JSON.stringify({
                radius: radius
            })
        }
    );

    loadHistory();
}

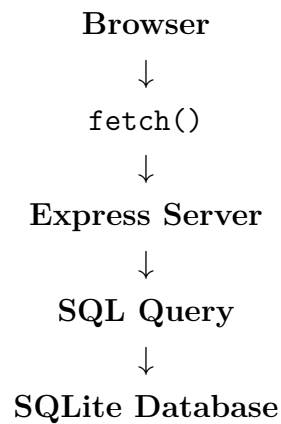
```

SQL ที่ใช้ใน Workshop

Operation	HTTP	SQL
Create	POST	INSERT
Read	GET	SELECT
Update	PUT	UPDATE
Delete	DELETE	DELETE

Complete CRUD Flow

ระบบที่สร้างใน Workshop นี้มี Flow ดังนี้



การทำงานแต่ละแบบคือ

- POST → INSERT INTO
- GET → SELECT FROM
- PUT → UPDATE SET
- DELETE → DELETE FROM

Checklist ก่อนส่งงาน

งานที่ 1 — Express + Frontend

- ☐ สร้าง Express Server ได้
- ☐ ใช้ `express.json()`
- ☐ ใช้ `express.static()`
- ☐ Frontend เรียก API ด้วย `fetch()`
- ☐ คำนวณพื้นที่วงกลมได้

งานที่ 2 — SQLite

- ☐ ติดตั้ง `sqlite3`
- ☐ สร้างไฟล์ `circle.db`
- ☐ สร้าง Table `circles`

- ☐ ใช้ INSERT บันทึกข้อมูล
- ☐ ใช้ SELECT อ่านข้อมูล
- ☐ ใช้ ORDER BY เรียงข้อมูล
- ☐ แสดง History บนหน้าเว็บ

งานที่ 3 — Complete CRUD

- ☐ ใช้ POST สำหรับ Create
- ☐ ใช้ GET สำหรับ Read
- ☐ ใช้ PUT สำหรับ Update
- ☐ ใช้ DELETE สำหรับ Delete
- ☐ ใช้ Route Parameter :id
- ☐ ใช้ SQL Parameter ?
- ☐ Refresh History หลังแก้ไขข้อมูล

งานที่นักศึกษาต้องส่ง

ส่ง Folder Project โดยอย่างน้อยมี

```
sql2-demo/
|-- app.js
|-- package.json
|
`-- public/
    |-- index.html
    |-- style.css
    `-- script.js
```

พร้อม Screenshot ที่แสดง

1. การคำนวณพื้นที่วงกลม
2. Calculation History
3. ผลหลัง Update หรือ Delete ข้อมูล

Frontend → Fetch API → Express → SQL → SQLite